QCon
全球软件开发大会

InfoQ 技术大会

AgentENV
面向智能体的执行环境

闪英迪

清华大学计算机系

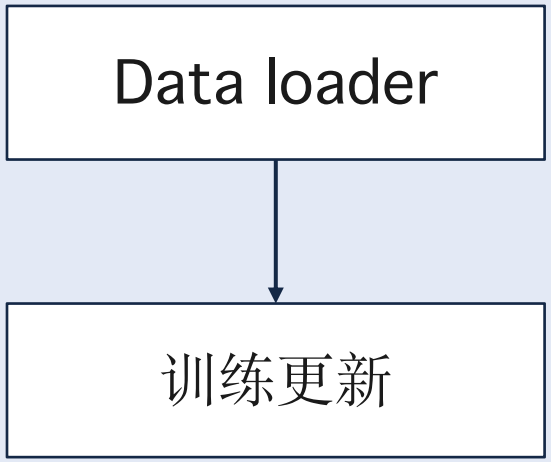


01

背景

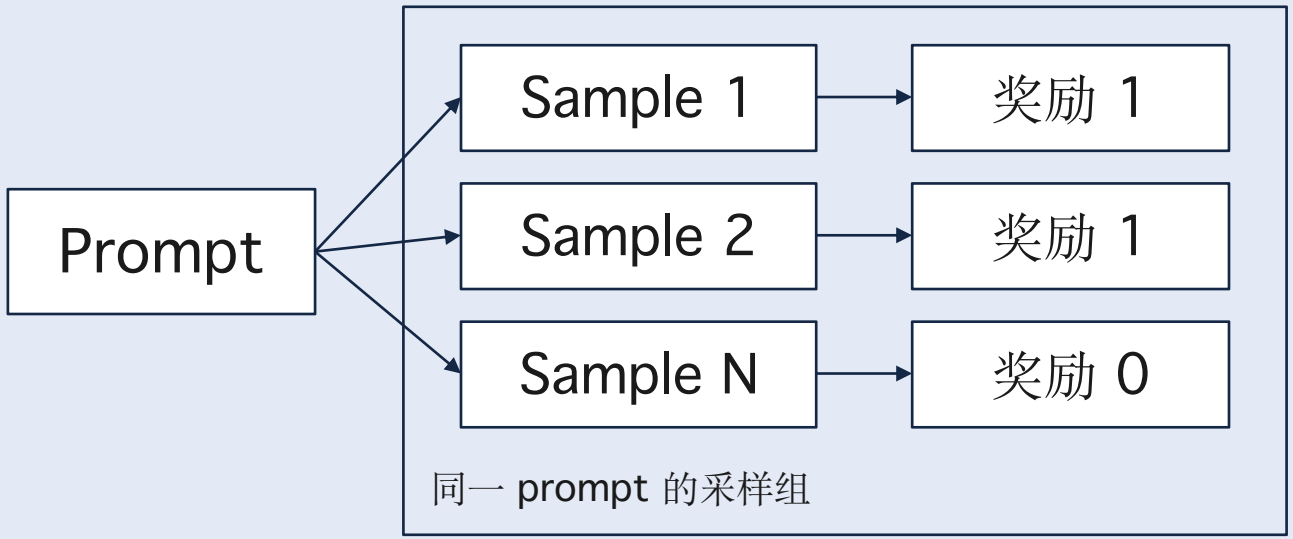
训练：Pre-training 与 SFT

- **Pre-training**：从大规模语料中学习预测下一个 token
- **SFT**：使用指令与回答示例，训练模型按要求完成任务
- 核心输入是已有数据



训练：RL 利用奖励改进模型

- 模型生成候选回答，通过外部奖励信号优化策略
- 以 GRPO 为例：同一 prompt 采样一组回答，比较组内相对奖励

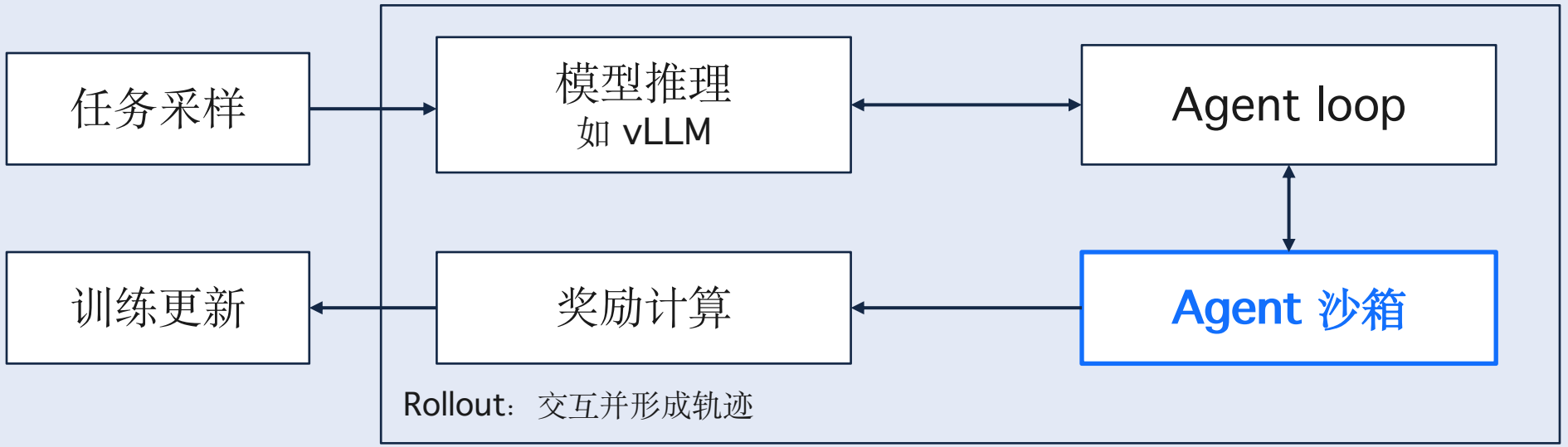


奖励信号可以来自

规则、测试程序或奖励模型（RM）

GRPO 用组内相对奖励估计 advantage，提升高奖励回答的概率

训练：Agent Rollout 依赖执行环境



- **数学**：用答案或规则验证结果
- **单轮代码**：在 OJ 中编译、运行并验证
- **Agent**：多轮工具调用，需要持续保留文件、进程与测试环境

题目数据与可执行环境

抓取 Issue 和 PR，可以得到题目与代码，还需要补齐运行环境

组成	包含什么	如何获得
Issue	需求、错误日志与复现步骤	GitHub Issue 正文
Repo	问题发生时的代码	仓库代码 + base commit
Env	编译、运行和测试所需依赖与配置	可以让 Agent 构建
Solution	参考修复代码（gold patch）	从关联 PR 的 diff 中分离
Test	验证修复所需的测试与执行方式	PR 测试修改 + Agent 补齐

Workspace 承载题目代码与构建好的依赖环境，供后续任务复用

AI Agent 需要自己的执行环境

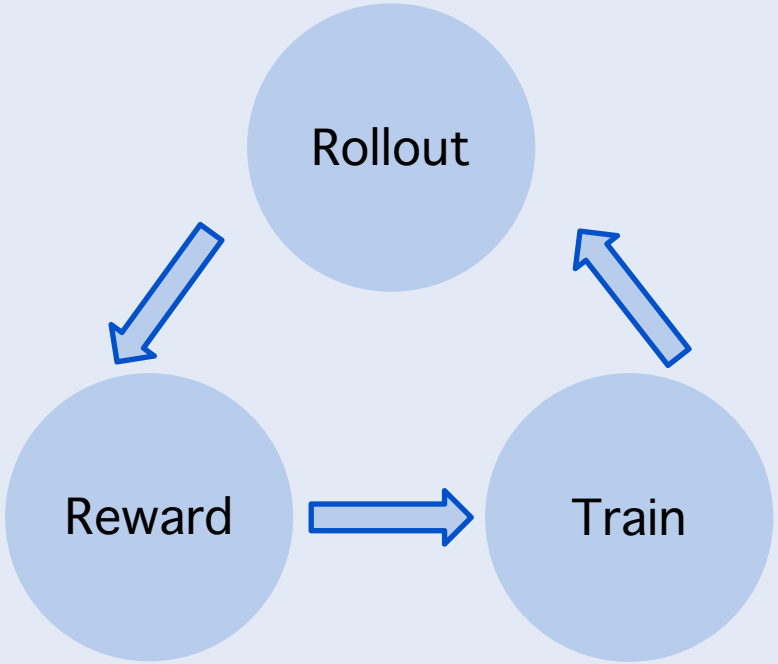
- AI Agent 除了训练，需要自己的执行环境（沙箱），还需要服务于：
  - **训练评测**：评估当前模型的能力
  - **在线服务**：Word、PPT 生成、图表生成、OpenClaw
- 然而，现有的沙箱平台难以满足实际生产的需求
  - **成本**高昂
  - **环境定义**不灵活
  - **镜像分发**效率低下

02

挑战

Agent 训练时的执行环境成本高昂

- 强化学习在进行并发Rollout 时，需要大量的CPU 和内存资源，厂商建议“32–128 并发 rollout 需配备 32–64 vCPU/GPU”
  - **DeepSWE**： 1000 核 / 64 GPU ≈ **16 vCPU/GPU**
  - **SWE-MiniSandbox**： 184 核 / 8 GPU ≈ **23 vCPU/GPU**



资源	用量	成本
GPU	128 卡	$ 640
申请 CPU	5859 vcpu * h * 0.04$	$ 234
申请 MEM	22.9 TB * h * 0.005$/(GB*h)	$ 117

一次 Code Agent RL 训练中Rollout 阶段各资源消耗情况

执行环境侧开销占比约 35%，是不可忽视的成本来源

Agent 推理时的执行环境成本高昂

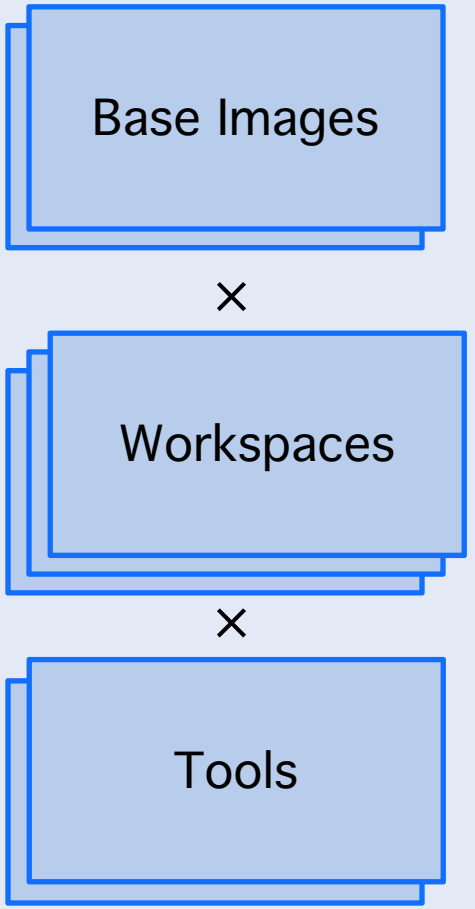
任务	Input Token	Output Token	LLM Time	Tool Time	DSV4-Flash	Lambda	Sbx占比
A	1,414,968	38,227	1434.1 s	123.4 s	$ 0.0535	$ 0.0519	***49.2 %***
B	571,240.0	8,213	861.6 s	23.0 s	$ 0.0196	$ 0.0295	***60.1 %***
C	3,005,124	17,149	1884.1 s	1562.2 s	$ 0.0957	$ 0.1149	***54.6 %***
D	2,382,438	46,942	2203.1 s	136.1 s	$ 0.0852	$ 0.0780	***47.8 %***
E	1,882,950	17,072	1973.4 s	310.9 s	$ 0.0617	$ 0.0761	***55.2 %***

使用DeepSeek V4 Flash 和 AWS Lambda 运行 SWE Bench 中5个任务的价格估算

Agent 执行环境在推理、训练过程中的成本高昂

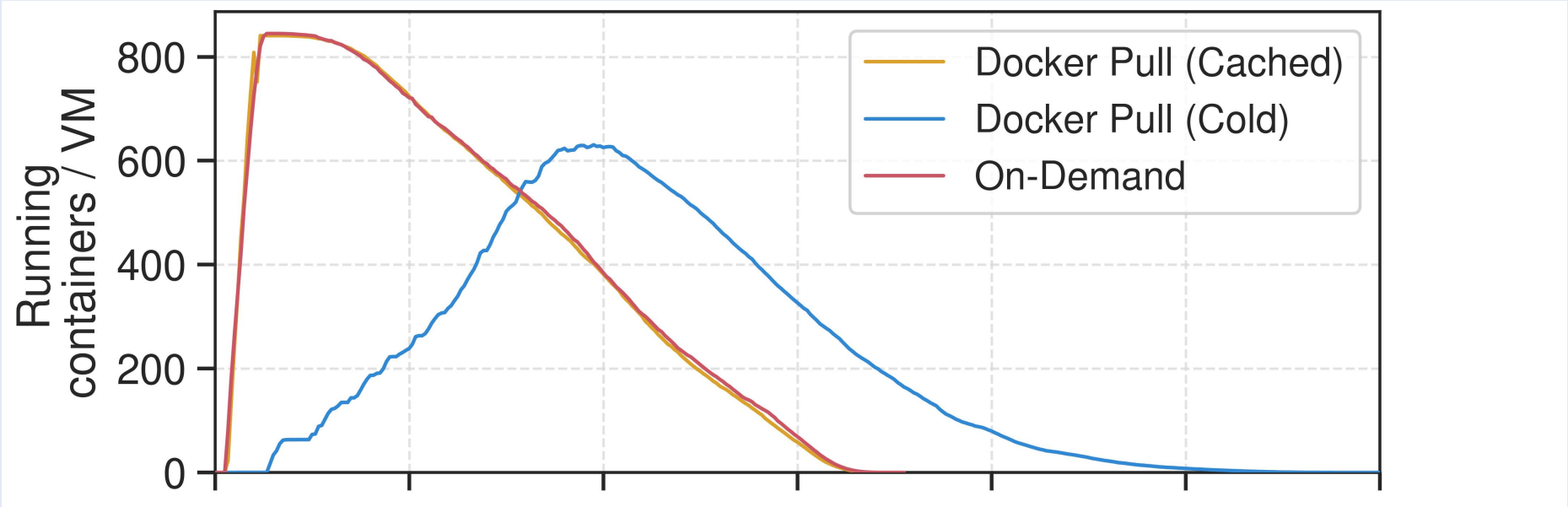
Agent 需要灵活的执行环境：单体模板无法承载环境组合

- **Agent 需要的执行环境是三种维度的组合:**
  - base(基础环境)× tool(辅助工具)× workspace(任务环境与依赖)
  - Kimi K3 的训练需要上百万的镜像
- **主流平台(以 E2B 为代表)的抽象是单体模板:一个 Dockerfile 线性构建为一个 template,沙箱只能从 template 整体启动**
  - **组合爆炸**：n 个 base × m 个工具版本 × k 个 workspace ⇒ 需维护 n×m×k 个模板
  - **更新耦合**：任一组件升级 ⇒ 所有含它的模板整体重建、重新发布



Agent需要高效的镜像分发

- **执行环境需要分发海量镜像**
  - 传统的 OCI 镜像不支持随机访问，必须全量下载并 unpack 后才能启动沙箱
  - 镜像分发会显著降低执行环境的运行效率



单步 Rollout 使用不同方案的对比

Agent 的行为可能破坏任务与环境

主动寻找捷径

训练和评估结果可能因此失真

- 探测控制面 socket 或日志，尝试伪造请求、读取残留答案
- 覆盖 /bin/bash，干扰后续命令执行
- 尝试用 XFS_IOC_SWAPEXT 绕过访问控制，结果损坏文件系统
- 经 Go module proxy 或新版软件包寻找现成实现

破坏执行环境

普通命令或执行错误也可能引发故障

- 全局 grep 读取 /proc/kpagecgroup，触发内核 bug
- 把面向目标 VM 的攻击命令误执行在自身容器内
- yes 持续输出，日志记录累积到数十 GB

使用容器方案，每天都有约 5% 的节点因 Agent 的破坏而宕机

02

AgentENV 总览

面向智能体的分布式执行环境

AENV：面向智能体的分布式执行环境

- **AgentENV**（简称AENV）是面向智能体的分布式执行环境
- 已服务于Kimi K3 训练，支撑 150 万个镜像与 5000 万个执行环境
- https://github.com/kvcache-ai/AgentENV
- 运行在近千节点上，支撑**数十万**沙箱并发运行，每天产生快照数据**1PB+**
- AgentENV 的存储后端已被 Deepseek 的DSec 所使用



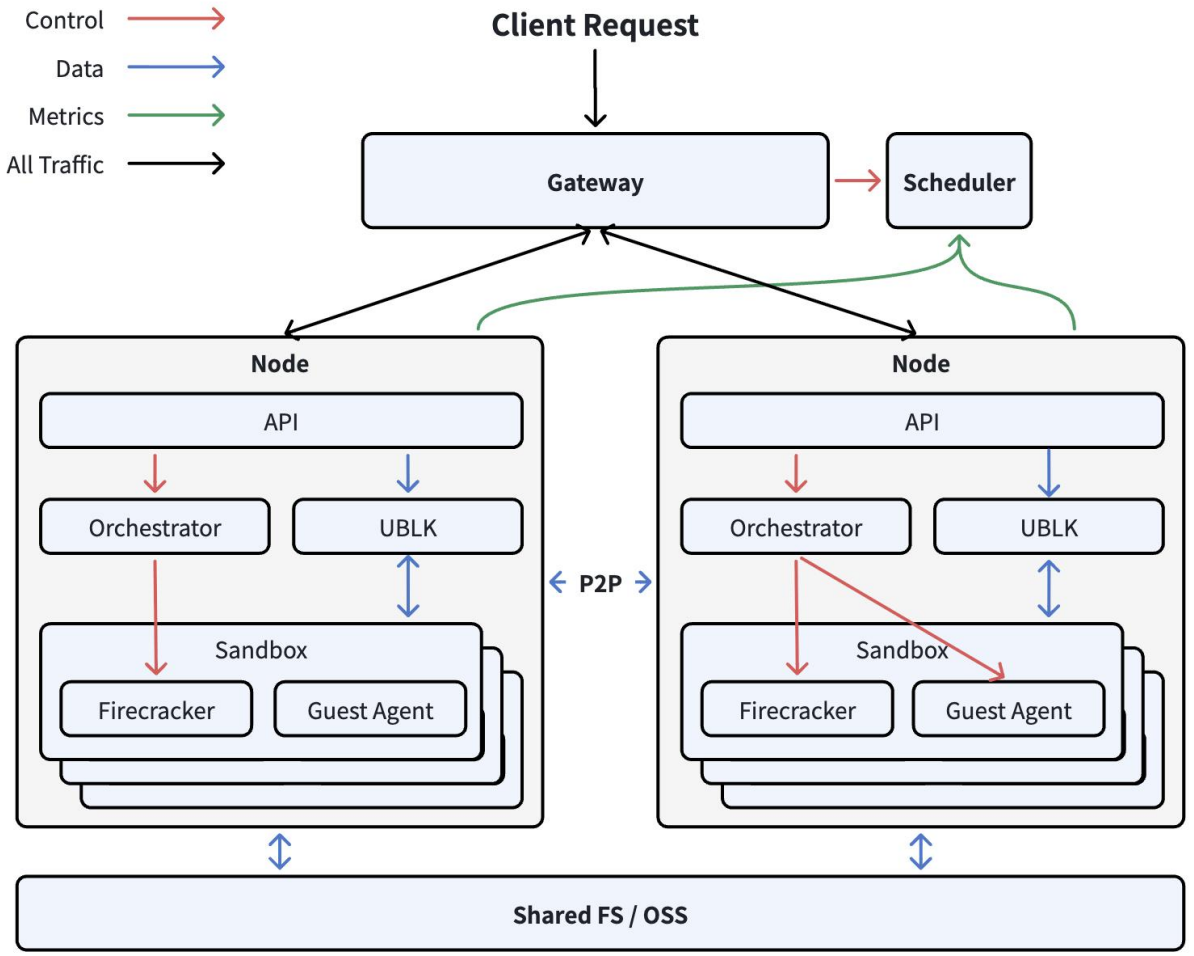


AgentENV

AENV 的总体优势

- **基于MicroVM 提供强隔离**
- **降低每个执行环境的部署成本**
  - **提升部署密度：** 减少内存、存储和运行时资源消耗
  - **提升资源利用率：** 暂停、持久化、恢复低活跃的执行环境
- **高可扩展性**
  - 数据存储在对象存储中，本地硬盘仅作为cache 和 buffer，可无限扩展
- **易于使用**
  - 兼容E2B，无需代码修改
  - 支持多容器镜像的组合，支持执行环境中嵌套 Docker
  - 无缝衔接对象存储，不需要设置复杂的分布式存储

AENV 的架构



- **Gateway**：请求转发，可无限横向扩展
- **Scheduler**：无状态的调度器，从各个节点收集CPU/内存等信息，决定调度目标
- **Node**：负责管理每个节点的执行环境
  - **Firecracker**：给每个执行环境独立的虚拟机
  - **Ublk**：给每个执行环境提供内存镜像与存储镜像
- **Shared FS/OSS**：负责数据的可靠存储

03

关键技术

底座：把环境状态放在对象存储上

方案	优点	问题
本地盘	访问延迟最低	状态绑定机器，单机放不下
云块存储	块语义，延迟低	创建挂载需数分钟，按预配置容量计费
对象存储	**自动扩容，按用量计费**	**对象不可变，访问延迟数十毫秒**

AENV 的做法：在对象存储上构建 VM 可用的块存储

- 镜像、磁盘、内存快照统一表示为不可变层 + 本地可写层
- OverlayBD 合成块视图，经 ublk + io_uring 交给 VM
- 热页预取 + 按需读取，把远端延迟挡在关键路径之外

底座：基于 ublk + overlaybd 的分层块存储



- **AENV** 基于ublk + overlaybd 实现了高性能的块存储
- 可将多个存储的修改叠为一个存储设备
- 每个存储层对应 Docker 镜像中 OverlayFS 的一层，可在本地或者远端，按需读取
- 远端的数据会在后台下载到本地
- 本地数据仅作为缓存，会通过 GC 进行回收
- 多个存储设备可共享底层 layer 的页面缓存

只读用到的数据，压缩后再传

问题： OCI 镜像须全量下载、解包，而运行时只用到 4%–13%

1. **VM 读取一个块**
   本地缓存命中则直接返回
2. **从对象存储读取压缩块**
   前台 256 KB，尽快返回
3. **按块解压，写入本地缓存**
   支持 zstd / lz4
4. **后台批量补齐**
   4 MiB 大块下载，不抢前台

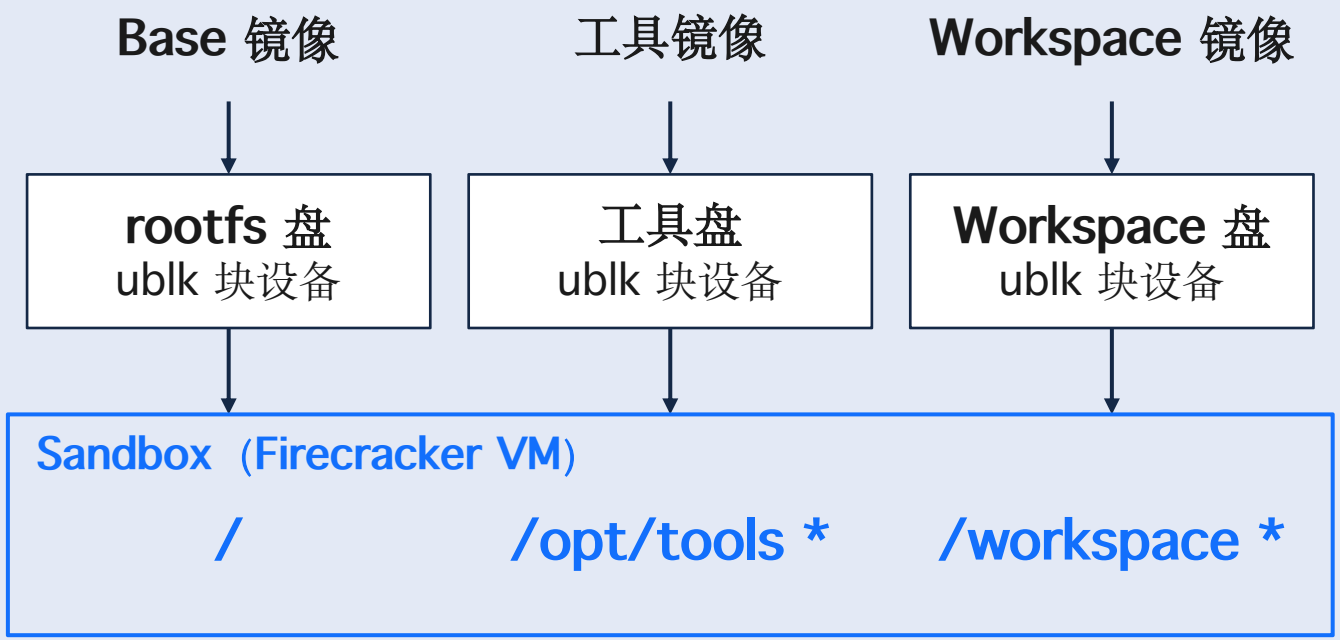
只搬运真正用到的数据

- 只用到镜像 4%–13% 的数据
- 前台小块、后台大块，共用缓存
- 镜像层与快照层都原生压缩
- 本地盘只做缓存，可无限扩展

启动无需全量下载和解包，准备工作量由实际访问范围决定

Attached Drive，一个沙箱挂载多个镜像

base × tool × workspace 组合爆炸，任一组件升级都要重建



组合在创建时完成

- 每类镜像独立构建、版本化
- 创建时选 base，再挂工具盘、数据盘
- 升级工具只换工具盘
- 多个沙箱共享只读镜像
- 冷启动：无需提前构建模板

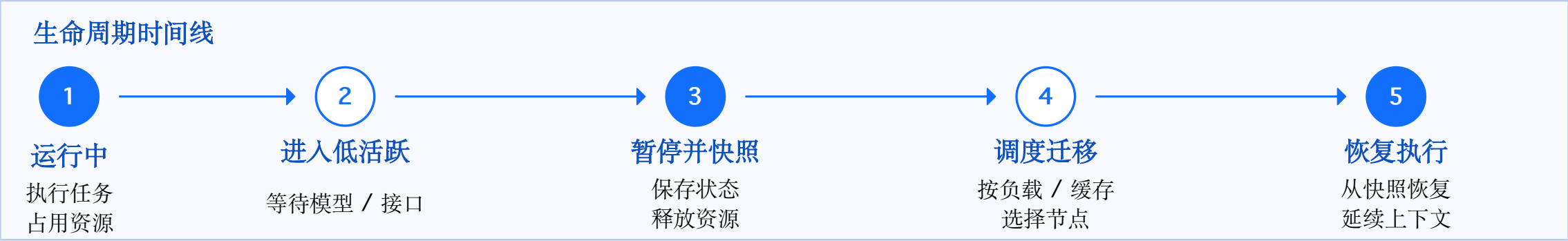
* 挂载路径由用户配置

n × m × k 个模板 → n + m + k 个镜像：哪个组件升级，就换哪块盘

AENV 通过复用提升部署密度

- **PageCache 占据了每个 VM 的绝大部分内存开销**
  - **将 Page Cache 放在Guset 端：**多个VM 读取的Page 存在冗余，结合 aenv 块存储，将Page Cache 从Guest OS 卸载到 Host，可显著降低内存消耗
  - **内存归还：**结合 DAMON 与 virtio-balloon，把 guest 内不活跃的 file-backed cache 或可回收页面归还宿主机
- **从同一 RootFS 启动的多个 VM的内存可复用**
  - 将每个镜像启动时所需的内存构建为一个 Memory Block Device，映射到 VM 中
  - 多个执行环境的内存通过OS 的mmap 机制实现自然共享

AENV 通过暂停、恢复、调度提高资源利用率



- AENV 通过 VM 的**自动暂停**，实现内存资源的自动释放
- 与 RL 框架协同：训练暂停时主动 pause，恢复后继续执行

AENV 实现极速的恢复

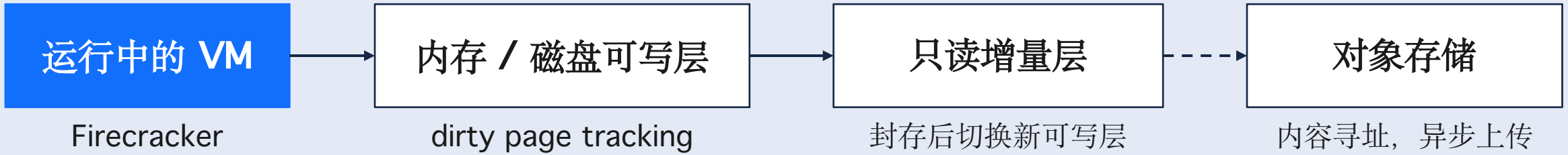


热页预取　　快照后异步回放，记录首次访问的页；恢复时批量读取

按需读取　　其余页缺页时再读：前台 256 KB，后台 4 MiB

40 ms 完成虚拟机本地启动，　100 ms完成虚拟机远程启动

AENV 实现高效的快照/暂停



增量 快照 = 父状态 + 增量，成本取决于变化量

hybrid 写入 追加 + 原地覆写，已被 OverlayBD 上游接受

80 ms 完成虚拟机快照（取决于内存修改，平均每GB增量内存需要 0.4s）

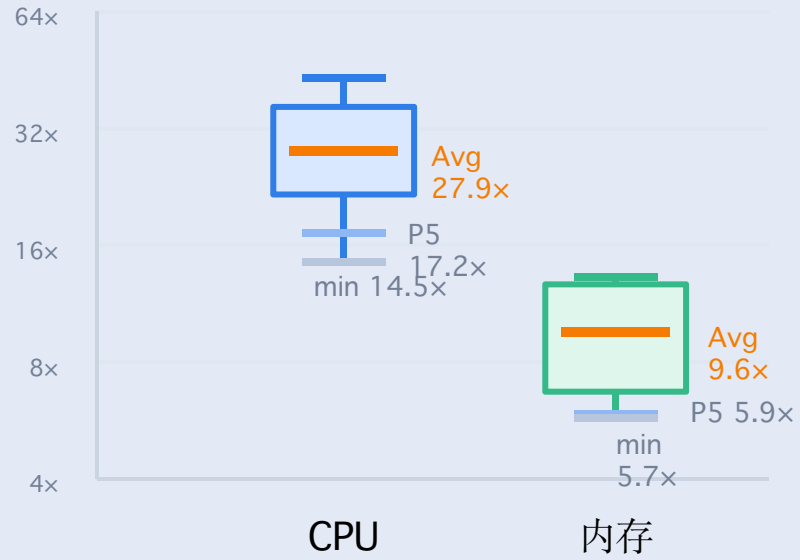
04

效果评估

资源复用能力直接转化为部署成本优势

统计数据来源：AENV 生产集群，覆盖 70 个节点；统计窗口内累计创建约 22.5 万个 sandbox，并基于这些 sandbox 的资源申请量与实际使用量计算申请/使用倍率。

分配-使用倍率 = 分配资源 / 实际使用 = 理论超售比例

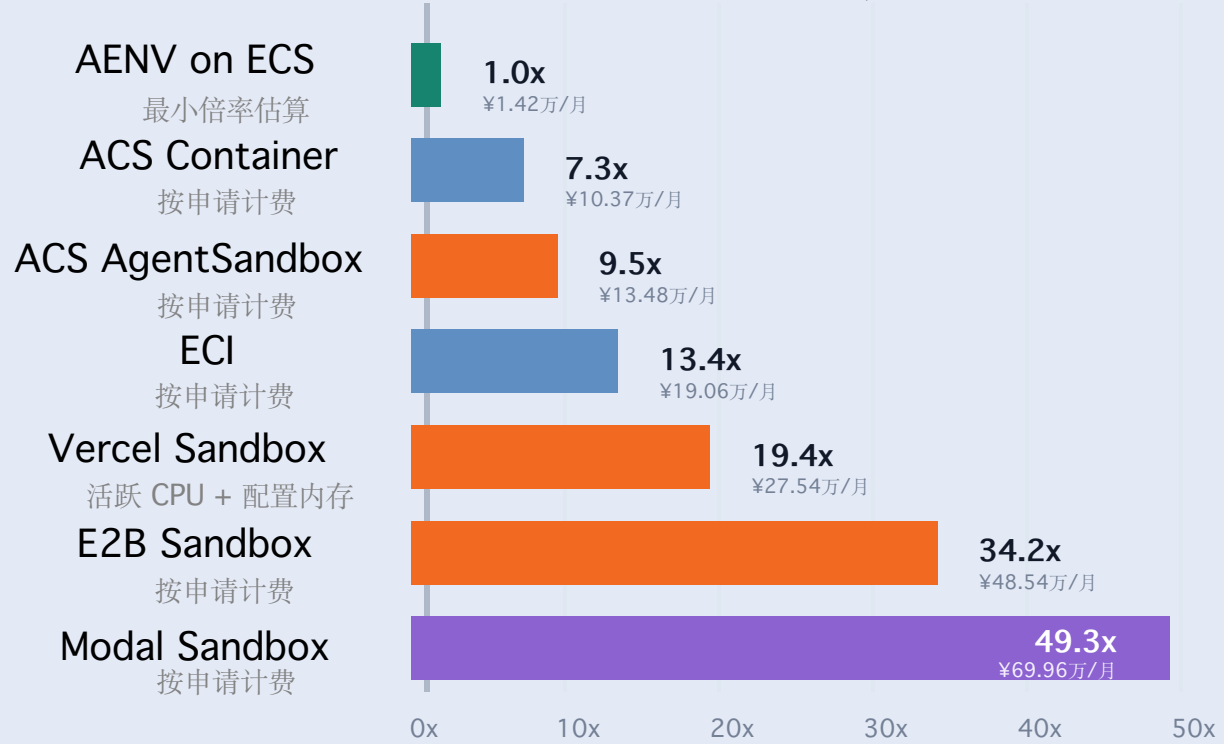


高负载窗口内的CPU/内存倍率

月度成本节省

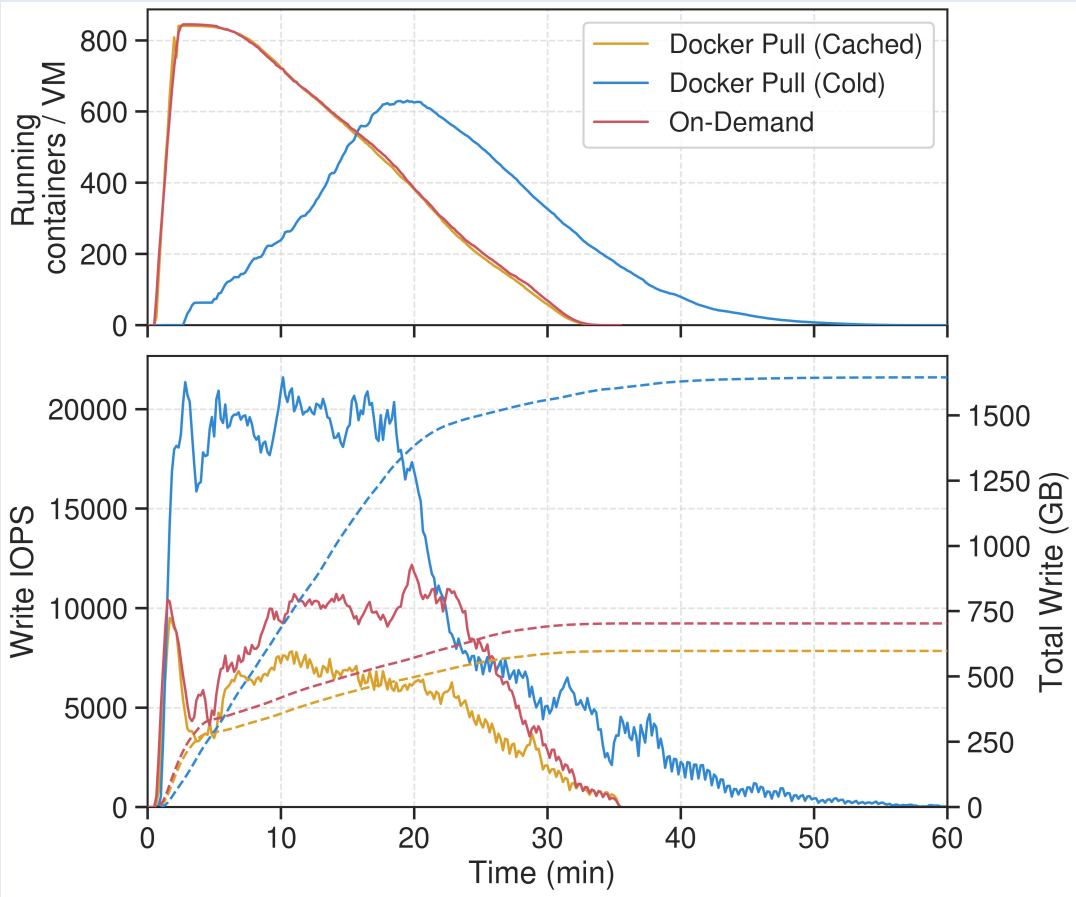
场景： 300 个 4 vCPU / 8 GiB sandbox 连续运行 720 小时

AENV在上述场景P5 约需 70 CPU / 408 GiB；最小约需 83 CPU / 418 GiB，均可由 1 台128C/512G ECS 承载



相对 AENV 的月成本倍率

镜像分发带来的性能优势



AENV 的镜像分发带来了显著的性能优势

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The Software

